Mentimeter

Laboratorul 6

Clase interne

Mentimeter

Laborator 6

Mentimeter

Ce sunt clasele interne?

→ clase declarate în interiorul altei clase

Mentimeter

Tipuri de clase interne

- clase interne normale (regular inner classes)
- clase anonime (anonymous inner classes)
- clase interne statice (static nested classes)
- clase interne metodelor (method-local inner classes) sau blocurilor

Mentimeter

O clasă internă

- se comportă ca un membru al clasei în care a fost declarată
- are acces la toți membrii clasei în care a fost declarată, inclusiv cei private
- poate fi public, final, abstract, private, protected, static

Mentimeter

Clase interne "normale"

- definite în interiorul unei clase
- pot fi accesate doar la runtime printr-o instanță a clasei externe
- compilatorul creează fișiere .class separate pentru fiecare clasă internă (dar care nu pot fi executate separat)
- în interiorul clasei interne putem accesa referința la clasa externă folosind this (<nume_clasa_parinte>.this)

Mentimeter

```java
interface Engine {
    public int getFuelCapacity();
}

class Car {
    class OttoEngine implements Engine {
        private int fuelCapacity;

        public OttoEngine(int fuelCapacity) {
            this.fuelCapacity = fuelCapacity;
        }

        public int getFuelCapacity() {
            return fuelCapacity;
        }
    }

    public OttoEngine getEngine() {
        OttoEngine engine = new OttoEngine( fuelCapacity: 11);
        return engine;
    }
}

public class Main {

    public static void main(String[] args) {
        Car car = new Car();

        Car.OttoEngine firstEngine = car.getEngine();
        Car.OttoEngine secondEngine = car.new OttoEngine( fuelCapacity: 10);

        System.out.println(firstEngine.getFuelCapacity());
        System.out.println(secondEngine.getFuelCapacity());
    }
}
```

Exemplu clase interne

Ce se afișează?

Mentimeter

The correct answer is: 11\n10\n

Mentimeter

```java
interface Engine {
    public int getFuelCapacity();
}

class Car {
    private class OttoEngine implements Engine {
        private int fuelCapacity;

        public OttoEngine(int fuelCapacity) {
            this.fuelCapacity = fuelCapacity;
        }

        public int getFuelCapacity() {
            return fuelCapacity;
        }
    }

    public OttoEngine getEngine() {
        OttoEngine engine = new OttoEngine( fuelCapacity: 11);
        return engine;
    }
}

public class Main {

    public static void main(String[] args) {
        Car car = new Car();

        Car.OttoEngine firstEngine = car.getEngine();
        Car.OttoEngine secondEngine = car.new OttoEngine( fuelCapacity: 10);

        System.out.println(firstEngine.getFuelCapacity());
        System.out.println(secondEngine.getFuelCapacity());
    }
}
```

Ce se întâmplă dacă facem clasa internă private?

Mentimeter

Observații

- constructorii devin privați
- putem crea instanțe de Car.OttoEngine prin intermediul interfeței Engine și a metodei getEngine
- folosim upcasting pentru a instanția clasa

Mentimeter

```java
interface Engine {
    public int getFuelCapacity();
}

class Car {
    private class OttoEngine implements Engine {
        private int fuelCapacity;

        public OttoEngine(int fuelCapacity) {
            this.fuelCapacity = fuelCapacity;
        }

        public int getFuelCapacity() {
            return fuelCapacity;
        }
    }

    public Engine getEngine() {
        OttoEngine engine = new OttoEngine( fuelCapacity: 11);
        return engine;
    }
}

public class Main {

    public static void main(String[] args) {
        Car car = new Car();

        Engine firstEngine = car.getEngine();
        System.out.println(firstEngine.getFuelCapacity());
    }
}
```

Clase interne private

Cum putem crea instanțe cu alte valori pentru fuelCapacity?

Mentimeter

The correct answer is: Parametru pentru metoda getEngine()

Mentimeter

Clase anonime

- utilizări reduse ale clasei => nu are sens să definesc o nouă clasă
- instanțe ale unei clase moștenite (sau a unei interfețe extinse)
- suprascriu anumite metode

Mentimeter

```java
interface Engine {
    public int getFuelCapacity();
}

class Car {
    public Engine getEngine() {
        return new Engine() {
            private int fuelCapacity = 11;

            public int getFuelCapacity() {
                return fuelCapacity;
            }
        };
    }
}

public class Main {

    public static void main(String[] args) {
        Car car = new Car();

        Engine firstEngine = car.getEngine();
        System.out.println(firstEngine.getFuelCapacity());
    }
}
```

Exemplu clase interne anonime

Mentimeter

return new Engine() {...}

creează o clază care
implementează interfață Engine,
conform următoarei implementări
+ instanțierea clasei folosind
new

Mentimeter

Observații

- reduce volumul codului + crește lizibilitatea acestuia
- obiectul este instanțiat imediat după return folosind new
- numele clasei instanțiate este absent (știm numai supertipul)

Mentimeter

Limitări

- nu au nume => nu știm cum să numim constructorii => nu vor avea constructori (diferiți de cel default)
- o clasă anonimă poate să implementeze o singură interfață (nu mai multe) sau să extindă o singură clasă, dar nu ambele în același timp
- nu pot folosi variabile declarate în metoda respectivă și nici parametrii metodei dacă nu sunt declarate cu final

Mentimeter

Accesul claselor interne la variabilele declarate în metodă

- numai dacă sunt finale sau effectively final
- effectively final: nu o declarăm cu final dar odată instanțiată, nu îi modificăm valoarea

Mentimeter

Expresii Lambda

→ funcții anonime

Mentimeter

```java
interface Engine {
    public int getFuelCapacity();
}

class Car {
    public Engine getEngine() {
        return () -> 11;
    }
}

public class Main {

    public static void main(String[] args) {
        Car car = new Car();

        Engine firstEngine = car.getEngine();
        System.out.println(firstEngine.getFuelCapacity());
    }
}
```

Exemplu expresie lambda

Mentimeter

```java
public class Main {
    public static void main(String[] args) {
        ArrayList<Integer> values = new ArrayList<>(Arrays.asList(1, 2, 3, 4, 5));

        values.removeIf((v) -> v % 2 == 0);
        values.forEach((v) -> System.out.println(v));
        values.forEach(System.out::println);
    }
}
```

Alte exemple expresii lambda

Mentimeter

Observații

- :: folosit pentru referențierea metodelor
- foarte multe funcții built in (spoiler PP sem 2): map, forEach, filter, reduce

Cum arată o lambda expresie care calculează suma a două numere a și b?

Mentimeter

The correct answer is: (a, b) -> a + b

Mentimeter

Clase interne statice

- nu avem nevoie de un obiect al clasei externe pentru a crea un obiect al clasei interne
- nu putem accesa câmpuri nestatice ale clasei externe din clasa internă

Mentimeter

```java
interface Station {
    public String getName();
}

class Car {
    static class GasStation implements Station {
        String name;

        public GasStation(String name) {
            this.name = name;
        }

        public String getName() {
            return name;
        }
    }
}
```

Exemplu clase interne statice



Mentimeter

```java
interface Engine {
    public int getFuelCapacity();
}

class Car {
    public Engine getEngine() {
        class OttoEngine implements Engine {
            private int fuelCapacity = 11;

            public int getFuelCapacity() {
                return fuelCapacity;
            }
        }
        return new OttoEngine();
    }
}
```

Clase interne în metode

Mentimeter

Clase interne în metode

- clasele sunt vizibile numai în interiorul funcției
- singurii modificatori care pot fi aplicați claselor: abstract, final

Mentimeter

```java
interface Engine {
    public int getFuelCapacity();
}

class Car {
    public Engine getEngine(int fuelCapacity) {
        if (fuelCapacity == 11) {
            class OttoEngine implements Engine {
                private int fuelCapacity = 11;

                public int getFuelCapacity() {
                    return fuelCapacity;
                }
            }
            return new OttoEngine();
        }
        return null;
    }
}
```

Clase interne în blocuri



Mentimeter

Explicații

- chiar dacă o declarăm numai în if, clasa se creează oricum
- why? numai la runtime o să știm pe care caz mergem
- clasa e vizibilă numai în blocul de if, nu și în rest (chiar dacă este creată)

Mentimeter

Moștenirea claselor interne

- constructorul clasei trebuie să se atașeze de un obiect al clasei externe
- mai complicată pentru că trebuie să inițializăm și legătura cu clasa exterioară

Mentimeter

```java
class Car {
    class Engine {
        public void getFuelCapacity() {
            System.out.println("I am a generic engine");
        }
    }
}

class OttoEngine extends Car.Engine {
    OttoEngine(Car car) {
        car.super();
    }
}

class Test {
    public static void main(String[] args) {
        Car car = new Car();
        OttoEngine ottoEngine = new OttoEngine(car);
        ottoEngine.getFuelCapacity();
    }
}
```

Exemplu de moștenire a unei clase interne

Mentimeter

Utilizarea claselor interne

- clasa nu mai are utilitate în alte zone ale programului sau nu vrem să fie accesibilă din exterior
- ascunderea implementării unei interfețe
- nu putem extinde mai multe clase, dar putem defini clase interne cu acces la obiectul clasei exterioare

Mentimeter

Demo